



Optimizing Pretrained Transformers for Autonomous Driving

Vasileios Kochliaridis

vkochlia@csd.auth.gr

Aristotle University of Thessaloniki

Thessaloniki, Greece

Evaggelos Kostinoudis

ekostino@csd.auth.gr

Aristotle University of Thessaloniki

Thessaloniki, Greece

Ioannis Vlahavas

vlahavas@csd.auth.gr

Aristotle University of Thessaloniki

Thessaloniki, Greece

ABSTRACT

Vision-based end-to-end driving systems have demonstrated impressive capabilities through the utilization of large Transformer architectures. More specifically, researchers have combined Transformers with Imitation Learning, in order to construct agents that learn to map navigation states to actions from large datasets created by experts. Although this approach usually works well, it relies on specific datasets and expert actions, and thus achieving limited generalization capability, which can be quite catastrophic in uncertain navigation environments, such as urban areas. To overcome this limitation, we further expand the training process of the agent by applying the Phasic Policy Gradient algorithm, a Deep Reinforcement Learning (DRL) method that improves its generalization capability by enabling it to explore and interact with the environment. We further enhance our approach by integrating a custom reward function that penalizes the weaknesses of the pretrained agent, alongside with additional DRL techniques to enhance its efficiency and accelerate convergence. Our experimental results in the CARLA simulation environment demonstrate that our approach not only achieves robustness in comparison to previous approaches, but also shows potential for wider application in similar navigation scenarios.

CCS CONCEPTS

• **Computing methodologies** → **Computational control theory; Markov decision processes; Sequential decision making; Image representations; Planning under uncertainty.**

KEYWORDS

Transformers, Deep Reinforcement Learning, Autonomous Navigation, Image Representations

ACM Reference Format:

Vasileios Kochliaridis, Evaggelos Kostinoudis, and Ioannis Vlahavas. 2024. Optimizing Pretrained Transformers for Autonomous Driving. In *13th Conference on Artificial Intelligence (SETN 2024)*, September 11–13, 2024, Piraeus, Greece. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3688671.3688778>

1 INTRODUCTION

Inspired by the recent advances in Natural Language Processing (NLP) achieved through large Transformer models, as highlighted

by Vaswani et al. [14], the field of Computer Vision (CV) has similarly embraced Transformer architectures. These architectures have been extensively applied to a variety of image recognition tasks [3] and have demonstrated exceptional capabilities in generalizing patterns in large datasets [6].

One of the more intricate challenges in the CV domain is the development of end-to-end navigation agents, which utilize multiple RGB cameras and various CV techniques to process and interpret visual information from a vehicle's surroundings, thereby enabling autonomous navigation [13]. To accelerate the development of such agents, researchers have combined Imitation Learning (IL) with Deep Learning, a process which involves training Deep Learning models to imitate expert driving behaviors, such as controlling the steering and speed, which they learn directly from millions of driving instances, generated by human drivers. Moreover, the integration of these systems with Transformer architectures has resulted in significant performance improvements [15].

Despite the advantages of imitation learning (IL) agents, their effectiveness is constrained, due to their reliance on the generated experiences and expert actions, as highlighted by Baker et al. [2]. More specifically, in order for the trained agents to perform efficiently, a comprehensive collection of experiences is required by the human experts for various driving scenarios. A lack of experiences in certain scenarios can result in the agent performing poorly. Therefore, if an agent does not efficiently explore its environment and relies solely on the generated experiences, it could significantly downgrade its generalization capability and overall performance. This limitation is particularly critical in autonomous navigation, especially within high-uncertainty environments, such as urban areas [13].

While IL often suffers from insufficient exposure to diverse driving situations during training, Deep Reinforcement Learning (DRL) is more immune to this problem. DRL methods have been proposed to tackle the generalization issues [13], where agents are trained through interaction and exploration with their environments. DRL agents are not told explicitly how to act by experts, but are evaluated by a reward function instead. For each state experienced, the agent chooses an action and receives a reward from its environment. The goal of the agent is to maximize the cumulative rewards received over its lifetime. By doing so, DRL agents develop robust navigation policies that can generalize well to diverse and unseen scenarios.

Although DRL has demonstrated satisfying performance across several navigation tasks, it is known to be less data efficient than IL, as well as time-consuming, making it challenging to use in the real world. To address these issues, Liang et al. [8] have proposed an approach, named CIRL, where policies are initially trained with IL and subsequently fine-tuned using DRL methods, and thus reducing the required training time. Despite this strategy has shown



This work is licensed under a [Creative Commons Attribution International 4.0 License](https://creativecommons.org/licenses/by/4.0/).

SETN 2024, September 11–13, 2024, Piraeus, Greece

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0982-1/24/09

<https://doi.org/10.1145/3688671.3688778>

promising results, it did not manage to surpass the performance of the State-Of-The-Art Transformer-based IL agents.

In this paper, we investigated the work of Liang et al. [8] and found that although CIRL has the potential to outperform IL agents, several overlooked factors led to the underperformance of their agent. First, the authors completely disregarded the use of Transformers in their policy architecture. Second, they employed an outdated DRL method, named Deep Deterministic Policy Gradient (DDPG) [9], which, while effective in some tasks, fails to achieve satisfactory performance in the challenges of autonomous navigation. Additionally, we examined more advanced and recent approaches, such as Proximal Policy Optimization (PPO) [10], which also failed to deliver satisfactory performance. We discovered that during the fine-tuning process with DRL, the policy tends to forget previously learned tasks from IL, resulting in slower training and a downgrade in the performance.

To effectively address these issues, we propose CILRL++, which integrates a Transformer-based architecture with IL and a more advanced DRL algorithm known for its exceptional performance in fine-tuning tasks, named Phasic Policy Gradient (PPG) [2]. To speed-up the fine-tuning process and enhance the agent's performance, we introduce a reward function that encourages the agent to focus on the weaknesses of the previously pretrained Transformer policy. Additionally, we further improve the policy by combining PPG with several DRL techniques that have proven effective in various DRL settings. Our contribution can be summarized as follows:

- Develop State-Of-The-Art end-to-end navigation systems for high-uncertainty environments.
- Highlight the potential of DRL-based fine-tuning methods to navigation tasks, with applicability to similar environmental settings.
- Effectively address the catastrophic forgetting issue of DRL policies during fine-tuning process.

The remainder of this paper is organized as follows. Section 2 presents all the important literature prior to this work. Section 3 covers thoroughly the CILRL++ approach. Section 4 presents the simulation environment, the experimental setup, the metrics, as well as presents and discusses the experimental results. Finally, Section 5 concludes this work and proposes future extensions.

2 RELATED WORK

Deep Deterministic Policy Gradient (DDPG), proposed by Lillicrap et al. [9] is a Policy-Gradient method which can operate in continuous action spaces. DDPG combines the Actor-Critic framework with observation normalization and policy noise, aiming to improve generalization capability and handle the exploration insufficiency of Policy Gradient algorithms [12]. Although DDPG achieved promising results in Atari environments, overestimated state values can easily lead to catastrophic divergence of the policy [7].

On the other hand, Proximal Policy Optimization (PPO) is a more recent algorithm, which is usually faster, more robust and more stable than DDPG [10]. There are two main variants of PPO: PPO-Clip (or Clipped Surrogate Objective) and PPO-Penalty, which incorporates KL-penalty. Both methods aim to clip the updates of the policy in each iteration, in order to constrain the new policy from going far off from the old policy, and thus tackle the catastrophic

forgetting issue of DDPG. Moreover, PPO also utilizes the Actor-Critic framework, in order to further increase the performance of the agent. Even though PPO is widely used due to its competitive results in combination with the ease of implementation, it displays a non-exploratory behavior over the course of training because of its tendency to exploit found rewards, since each updated policy becomes less random. Moreover, the parameter sharing between Actor and Critic networks could lead to undesired policy updates, since the objective optimization of the one interferes with the objective optimization of the other.

Phasic Policy Gradient (PPG) is a recent extension of PPO, proposed by [4], which separates the training of the Actor and the Critic networks, while maintaining parameter sharing between the networks. In PPG, the training procedure involves two alternating phases, named Policy Phase and Auxiliary Phase. During the policy phase, the Actor network is trained using PPO, while in Auxiliary Phase, important features are distilled from the value function into the Actor network, to improve training in future policy phases. Although PPG is similar to PPO, the inclusion of the periodic Auxiliary Phases of PPG has showcased more impressive results and sample efficiency in comparison to PPO and DDPG, especially in fine-tuning tasks [2].

In the field of end-to-end autonomous driving, Codevilla et al. [5] proposed CIRLS, an IL approach which utilizes a Deep Neural Network (DNN) model to estimate the vehicle controls. CIRLS was trained using a limited dataset of RGB camera images paired with expert vehicle control demonstrations within the CARLA simulator. The architecture of the model is composed of a base model and a Fully-Connected (FC) network. A pretrained ResNet34, with frozen weights, serves as the base model, responsible for extracting features from the input images. These features are subsequently fed into the FC network, which is trained to estimate the expert's controls, including steering, throttle, and brake. Despite achieving promising results, the authors acknowledged that their approach is prone to over-fitting, dataset biases, and generalization issues.

Xiao et al. [15] addressed key limitations of CIRLS by introducing CIL++ (Figure 1), a robust vision-based end-to-end driving system featuring a visual Transformer-based architecture that functions as a mid-level Attention mechanism for the observed images. Moreover, CIL++ enhances its input state by including the positional embeddings of the images, as well as the vehicle control embeddings, as show in Figure 1. Similar to CIRLS, CIL++ also employs a ResNet34 network as the backbone to extract valuable features from the input images. However, instead of directly passing these features to the FC network, the extracted feature maps first go through Self-Attention blocks, which further process the features generated by ResNet34 before passing them into the FC network. This architectural enhancement enables CIL++ to achieve State-Of-The-Art performance in several complex towns within the CARLA simulator. Nonetheless, this approach still faces generalization issues, particularly when deployed in previously unseen towns in the CARLA simulator.

CIRL was proposed by Liang et al. [8] to address the generalization issues faced by IL-based agents. In their approach, they constructed several expert observations to train a policy model to estimate the expert's actions. Then, they applied the DDPG algorithm to fine-tune the policy model by allowing it to explore

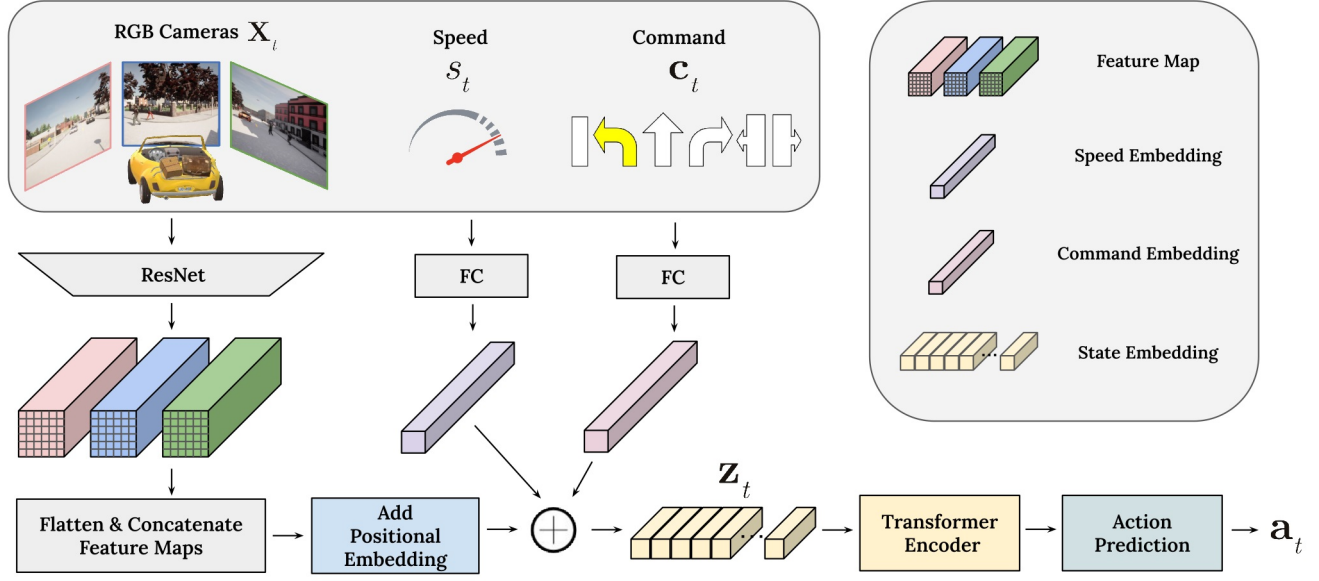


Figure 1: Overview of the CIL++ architecture.

the environment, thereby enhancing the model’s generalization capability. Although this work showed promising results, it did not outperform CIRLS or CIL++ in the evaluated performance metrics. This may be attributed to the outdated nature of the DDPG algorithm, which quickly tends to deviate from the original policy in cases of state value overestimation. Additionally, their approach lacks Transformer models, which have proven highly efficient in image recognition tasks and are a key component in the CIL++ approach.

To conclude our literature review, various attempts have been made to develop end-to-end driving systems. However, these systems often either underperform or face several generalization issues in complex environments. In our work, we aim to address these challenges by a) incorporating Transformer blocks into the policy model, b) Apply advanced DRL techniques to fine-tune the pretrained policy model, and c) introduce a reward function that enhances the policy’s training towards scenarios where it previously exhibited weaknesses.

3 METHODOLOGY

The proposed methodology aims to optimize the vehicle control by combining three modules. The first module, named Perception Module (PM), is responsible for the camera placement and image collection. The second module, named Imitation Learning Module (ILM) uses a transformer model to estimate the vehicle control, given the current vehicle’s state. The goal of the Third module, named Fine-Tuning Module (FTM), is to fine-tune the pretrained policy model generated by ILM, through DRL, increasing its generalization capability and performance in various driving scenarios. Our approach leverages the strengths of both paradigms to achieve

robust and adaptive vehicle control in diverse environments. Below, we outline the detailed steps of our methodology (Figure 2), emphasizing the technical aspects behind each component.

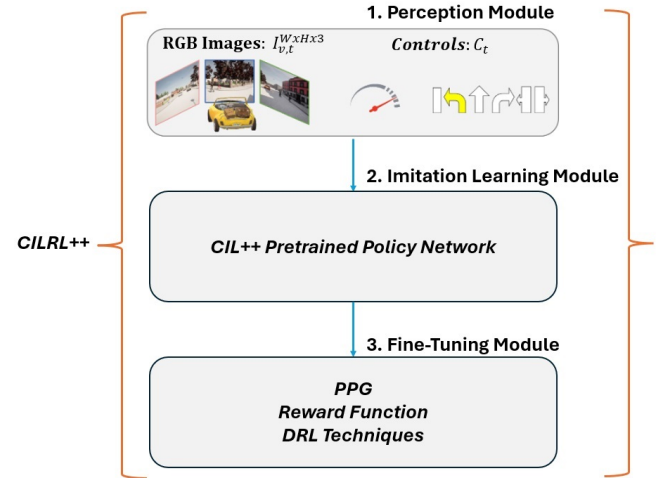


Figure 2: Overview of the unified CILRL++ architecture.

3.1 Perception Module

To simulate human-like perception, we deploy three RGB cameras on the vehicle, each with a 60-degree horizontal field of view (HFOV). The cameras are strategically positioned to cover the left, center, and right views, collectively providing the agent with a 180-degree HFOV. This setup mimics the visual range of human drivers and ensures comprehensive environmental perception [15]. The left

camera captures the peripheral left view, the center camera focuses on the front view, and the right camera monitors the peripheral right view. These three perspectives are crucial for making informed driving decisions, such as lane changes, obstacle avoidance, turning and traffic light detection.

Given a video sequence of N images, each RGB camera provides an image, denoted as $I_{v,t}^{W \times H \times 3}$, $v \in \{left, front, right\}$ and $t \in \{1, 2, \dots, N\}$, where W and H define the width and height of the images respectively. Because the size of image-adjusted Transformer architectures can become extremely high, we scale the resolution of the captured images into $W \times H = 300 \times 300$ pixels. Finally, the PM captures the current vehicle's control, denoted as C_t , which is passed into the policy model alongside with the captured images.

3.2 Imitation Learning Module

Instead of training from scratch a policy model using Imitation Learning, we use a pretrained CIL++ model, which has already been trained in a driving dataset, achieving State-Of-The-Art results in various scenarios. CIL++ takes as input the current images from the three cameras and the current control signals captured by the PM and estimates the next set of control signals c_{t+1} , which include the subsequent throttle and steering actions.

Next, we analyze the performance of CIL++ in various driving scenarios using the CARLA 0.9.15 Leaderboard module. Although CIL++ demonstrates robust performance in some situations, it exhibits unstable driving behavior in highly complex scenarios, such as those with dense populations and vehicles, as well as in sharp turns. Additionally, CIL++ sometimes fails to drive in a straight line, unnecessarily turning the steering wheel left and right. To address these issues, we construct a reward function, described later in this section, which is used by the FTM to fine-tune these weaknesses.

3.3 Fine-Tuning Module

The objective of FTM is to further optimize the CIL++ model by utilizing DRL methods, allowing CIL++ to improve its performance through exploration and exploitation with the simulation environment. To enable the utilization of DRL, we formulate the fine-tuning process of CIL++ as a Markov Decision Problem (MDP), with a 5-tuple $M = (S, A, p, \gamma, R)$, where S and A denote the state space and action space respectively, p is the environment dynamics function, γ is the discount factor and R is the reward function [12]. At each timestep t , the CIL++, which is defined as the DRL agent, generates an action a_t from a state s_t , receives a reward R_{t+1} and transits into the next state s_{t+1} . The goal of the DRL agent is to learn an optimal policy π_θ that maximizes the discounted cumulative rewards G_t , as described in Equation 1.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^T \gamma^k R_{t+k+1} \quad (1)$$

State Space (S). In this framework, the state s_t represents the observed environment through the camera inputs and vehicle controls, defined as a tuple of $(I_{v,t}^{W \times H \times 3}, C_t)$. Since the camera inputs are RGB images, each pixel can have values from 0 to 255. We normalize these values to a range of 0 to 1 by dividing by 255.0.

Action Space (A). The action a_t denotes the vehicle's control in the next timestep $t+1$, defined as a tuple of (th_{t+1}, st_{t+1}) , which represent the throttle and steering values of the next timestep. Throttle control values range from -1.0 to 1.0, with negative values indicating deceleration, while positive values indicating acceleration. Steering control values also range from -1.0 to 1.0, with negative values indicating a left steer, positive values indicating a right steer, and 0 indicating a straight path.

Reward Function (R). At each timestep t , the agent is rewarded based on the episode state. If the route of the episode is terminated then, the reward can be described as:

$$R_{t+1} = \begin{cases} c_{success} & \text{Route is completed,} \\ -c_{fail} \cdot (1 + speed_{t+1}) & \text{Route not completed} \end{cases} \quad (2)$$

where $c_{success}$ and c_{fail} are small constants in range $[0.0, 1.0]$ and $speed_{t+1}$ is the vehicle's speed right after the episode termination signal. During training, it was observed that speed is a crucial factor when a route is not completed. This reward function encourages the agent to finish the episode while also avoiding vehicle and pedestrian collisions, as well as running stop signs and red lights.

On the other hand, if the episode has not terminated, then the reward function can be expressed as the sum of smaller rewards, defined as $R_{t+1} = r_{lat_{t+1}} + r_{speed_{t+1}} + r_{angle_{t+1}}$, with each reward focusing on a specific weakness of the CIL++ approach. More specifically:

- $r_{lat_{t+1}}$: Is responsible for the lateral control of the vehicle, which keeps it inside the boundaries of the target lane and can be defined as:

$$r_{lat_{t+1}} = c_{lat} \cdot distance_{t+1} \quad (3)$$

where c_{lat} is a constant which controls the magnitude of the reward and $distance_{t+1}$ expresses the distance of the vehicle from the center of lane.

- $r_{speed_{t+1}}$: Is responsible for the longitudinal control of the vehicle, providing the agent with proper information about the desired speed of the vehicle. More specifically, if there are no obstacles, as well as traffic signs and traffic lights in front of the vehicle, the desired speed is the maximum speed. Otherwise, the desired speed decreases relative to the distance of these obstacles as follows:

$$r_{speed_{t+1}} = c_{speed} \frac{1 - c_{slope} \cdot (speed_{t+1} - DS_{t+1})}{c_{max_speed}} \quad (4)$$

where, c_{speed} is a constant which weights the speed reward, c_{slope} controls the range of the reward, DS_{t+1} is the desired speed of the agent and c_{max_speed} is the maximum allowed speed of the agent.

- $r_{angle_{t+1}}$: The purpose of this reward is to discourage the agent from oscillating around the route's center line, which was observed to be a major flaw of CIL++ approach. It is defined as:

$$r_{angle_{t+1}} = c_{angle} \cdot DA_{t+1} \quad (5)$$

where c_{angle} is a constant which controls the magnitude of the reward and DA_{t+1} is the absolute distance in radians between the direction of the vehicle and the route.

Algorithm. To fine-tune the pretrained policy model, we employ the PPG algorithm, described in Section 2. PPG is selected over other techniques like DDPG or PPO due to its superior performance in fine-tuning tasks and its ability to mitigate catastrophic forgetting. Moreover, we found crucial to incorporate a KL-divergence loss as an additional penalty term to prevent the new policy from diverging excessively from the original pretrained policy. This constraint ensures that the fine-tuning process retains the beneficial aspects of the initial policy while addressing its previous weaknesses. An illustration of CILRL++ approach in Carla environment is presented in Figure 3.

Other DRL Techniques. To further enhance the fine-tuning process and speed up the convergence time of PPG, we integrate several additional DRL techniques, as highlighted by Andrychowicz et al. [1]. More specifically, we include:

- **Observation Normalization:** Standardizes the input data to improve learning efficiency.
- **Value Function Normalization:** Prevents the value-function loss from having gradients that depend greatly on the magnitude of the reward by normalizing the value-function target. This is accomplished by subtracting the mean and dividing by the standard deviation, which are estimated through a moving average.
- **Action Clipping:** Limits the range of control actions to ensure smooth vehicle maneuvers.
- **Mini-Batch Shuffling:** Randomizes the order of training samples to reduce overfitting and improve generalization.
- **Generalized Advantage Estimation (GAE):** Combine the value estimations from multiple timesteps, leading to more accurate policy updates.
- **Advantage Normalization:** Normalizes the advantage values $A(s_t, a_t)$, which further improves the policy updates.

4 EXPERIMENTS & RESULTS

Our experimental setup has three main objectives: a) to investigate the performance and generalization capability of the CIL++ approach in various real-world driving challenges included in the CARLA leaderboard, b) to explore the impact of DRL on further refining and improving the CIL++ by training it in random CARLA environments, and c) to evaluate the efficacy of CILRL++. To train CILRL++, we employ random routes from towns 1 to 10 with randomly placed vehicles and pedestrians, except for towns 7 and 8, which are only available in the CARLA Leaderboard. A visualization of two of the towns that the agent is trained on is presented in Figure 4. We provide the code from the experimental setup, which includes the simulation setup and the implementation of CILRL++.

To evaluate each method, we use its average performance in 25 routes, which the agents don't have access during training. Each route has varying weather conditions and a specific scenario, including:

- Lane merging.
- Lane changing.
- Negotiations at traffic intersections.

- Negotiations at roundabouts.
- Handling traffic lights and traffic signs.
- Yielding to emergency vehicles.
- Coping with pedestrians, cyclists, and other elements.

An example of several evaluation scenarios is demonstrated in Figure 5. Finally, the evaluated methods are the following:

- (1) **CIL++.**
- (2) **CILRL** (Fine-tuning using PPG, without the KL-Divergence and the additional DRL techniques).
- (3) **CILRL++**

4.1 Metrics

The driving ability of an agent can be characterized by multiple metrics. Leaderboard employs a set of metrics that help understand different aspects of driving, which are described as below.

- (1) **Avg. Driving Score:** The main metric of the Leaderboard. The Driving Score is defined as $R_i \cdot P_i$, where R_i expresses the route completion percentage and P_i the infraction penalty.
- (2) **Avg Route completion:** The percentage of the completed route distance.
- (3) **Avg Infraction Penalty:** This score starts with an ideal 1.0 base score, which is reduced each time the agent commits an infraction.
- (4) **Collisions with Vehicles:** The number of collisions with other vehicles.
- (5) **Collisions with Pedestrians:** The number of collisions with walking pedestrians.
- (6) **Collisions with Layout:** The number of collisions with the environment (e.g. buildings, pavement, etc.).
- (7) **Red Light Infractions:** The number of red lights which are ignored by the agent.
- (8) **Stop Sign Infractions:** The number of stop signs which are ignored by the agent.
- (9) **Off-Road Infractions:** The number of times where the agent escaped its target lane.
- (10) **Route Deviations:** The number of times where the agent deviated from the desired route.
- (11) **Route Timeouts:** The number of times where the agent failed to complete its route in time.
- (12) **Agent Blocked:** The time that the agent did not make any actions.

When evaluating the performance of an agent, the first three metrics should be maximized, while the remaining metrics should ideally be 0.0.

4.2 Hyperparameter Tuning

Similar to PPO, the hyperparameters of PPG has a large combinatorial space, making an exhaustive search infeasible, so we manually conducted a limited search on the most sensitive. More specifically, we set the image sizes to 300x300 pixels, in order to achieve a balance between capturing sufficient visual features and accelerating the training process. Moreover, we set the clipping factor (ϵ) of PPG to 0.2, chosen from a range of $[0.1, 0.3]$, and the KL-divergence penalty factor to 0.1. The discount factor (γ) is set to 0.99, which

¹https://github.com/EKostinoudis/Carla_Autonomous_Driving

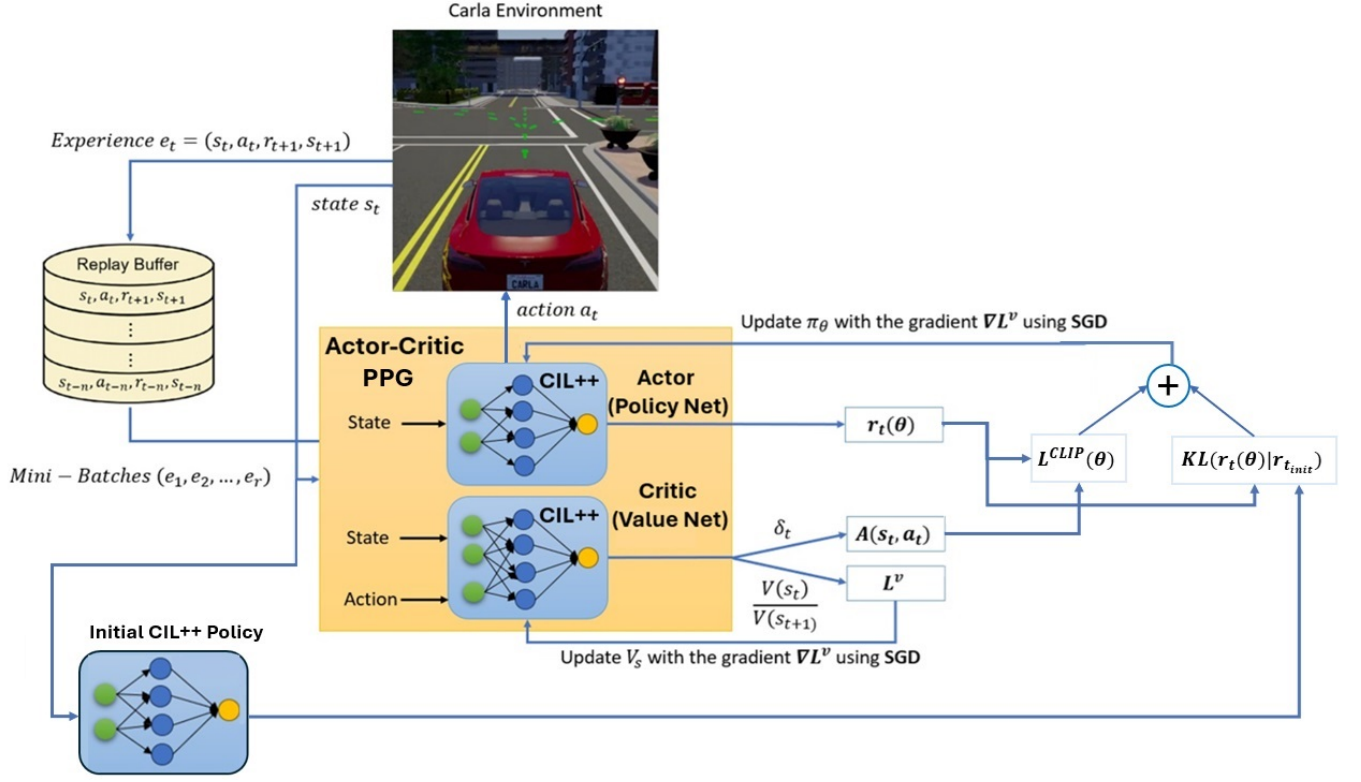


Figure 3: Illustration of the CILRL++ architecture in Carla Environment.



Figure 4: Illustration of Town10HD and Town01 of CARLA simulator, which are employed during fine-tuning process.



Figure 5: Illustration of Evaluation Scenarios.

is a typical value for DRL problems. PPG was trained on mini-batches of 128 samples, with a total batch size of 6144, due to low computational resources.

The Actor and Critic networks are trained for 1 epoch per batch using the Adam optimizer, as suggested in the original work of PPG. We fine-tuned the learning rate of Adam, which is set to 0.0004, chosen from a range of [0.0001, 0.001]. Moreover, we disable entropy regularization, as we found out that it has almost no impact in PPG’s policy updates and that PPG already provides the agent with enough exploration. Furthermore, we use the pretrained CIL++ model as the Actor and the same architecture for the Critic network.

Finally, we also fine-tune the reward function weights, as shown in Table 1.

4.3 Results

The results of the evaluation experiments for each method are presented in Table 2. It can be observed that CILRL++ outperforms both CIL++ and CILRL in most evaluation metrics, with CILRL having the worst performance overall, particularly in the Driving Score and Route Completion metrics. Although CIL++ achieved satisfactory performance and outperformed the CILRL approach, it

Table 1: CILRL++ Configuration

Parameter	Value
Clipping factor (ϵ)	0.2
KL-Divergence penalty factor (p)	0.1
Discount factor (γ)	0.99
Minibatch size	24
Train batch size	6144
Epochs (e)	1
Optimizer	Adam
Learning Rate (lr)	0.0004
Actor network	CIL++
Critic network	CIL++
Training Steps	1M
$c_{success}$	1.0
c_{fail}	-1.0
c_{lat}	-0.02
c_{speed}	0.05
c_{slope}	2.0
c_{angle}	-0.1
$c_{maxspeed}$	30.0

only excelled in the Route Deviation metric. Finally, all three agents learned to avoid vehicle collisions in all scenarios.

The fact that CILRL performs worse than its original policy, which is CIL++, highlights the importance of employing the KL-Divergence penalty and additional DRL techniques during the fine-tuning process. During our investigation, we discovered that applying a DRL technique, such as PPG, to fine-tune CIL++ can lead to several issues. As the training progresses, the agent tends to forget originally learned tasks, beginning to deviate from routes, becoming hesitant to make actions, and starting to collide with pedestrians and other elements of the environment’s layout.

On the other hand, the KL-Divergence penalty enables the agent to fine-tune several driving details that the CIL model lacks using the provided reward function. Additionally, it prevents the agent from deviating significantly from the original pretrained policy, thereby retaining several previously learned driving skills. However, it is important to note that the penalty factor p requires tuning, as it does not always yield satisfying results.

Finally, we scaled CILRL++, allowing it to train for 13 million steps in the CARLA environment. The results of the training, in terms of learning the optimal behavior according to the defined reward function, are presented in Figure 6. Although the rewards increase slowly per training step, it is evident that they steadily rise over time.

4.4 Action Explainability

In the development of autonomous driving systems, ensuring that the model’s decisions are explainable and justifiable is crucial for the safety of the passengers, as well as the reliability of the system. In our approach, we enhance the explainability of the DRL agent by employing the Grad-CAM method, as proposed by Selvaraju et al. [11]. The DRL model, which processes input images and outputs driving actions, utilizes a ResNet architecture as its backbone. By

Table 2: Evaluation Table of a) CIL++, b) CILRL and c) CILRL++ using CARLA Leaderboard.

Score	CIL++	CILRL	CILRL++
Avg. Driving Score (%)	2.593	0.092	10.019
Avg. Route Completion (%)	10.932	0.296	14.484
Avg. Infraction Penalty	0.404	0.467	0.654
Collisions with Vehicles	0.0	0.0	0.0
Collisions with Pedestrians	256.214	2759.687	52.842
Collisions with Layout	411.010	14496.833	255.232
Red Light Infractions	9.340	0.0	7.975
Stop Sign Infractions	5.767	846.623	0.0
Off-Road Infractions	253.666	1457.567	186.474
Route Deviations	0.0	144.636	76.737
Route Timeouts	0.0	330.962	0.0
Agent Blocked	398.996	10985.675	222.645

applying Grad-CAM, we can visualize the regions in the input image that the model focuses on when making decisions. Specifically, we extract gradient information from the final layer of the ResNet model to generate heatmaps that highlight the areas of the image that contribute most to the model’s actions.

In our experiments, we also employed a comparative analysis with the CIL++ and CILRL frameworks. The heatmaps generated using Grad-CAM for both models provided significant insights into their decision-making processes. Our DRL model demonstrated a strong focus on relevant regions of the input images, as demonstrated in Figure 7, which are critical for safe driving and navigation. The enhanced focus that is presented by our approach suggests that our agent has effectively learned to prioritize essential features in the driving environment, thereby enhancing its reliability and safety in real-world scenarios.

Contrastingly, the ResNet models from the CIL++ and CILRL frameworks exhibited a less coherent focus in the generated heatmaps, which resulted in wrong decisions. More specifically, the highlighted areas in the images are often dispersed and seemed to target random locations. Such behavior can potentially lead to suboptimal or unsafe driving decisions, as the model may not consistently recognize crucial features like traffic lights, road signs, or pedestrians.

5 CONCLUSION

In this paper, we introduce CILRL++, a DRL-based method for fine-tuning pretrained policies for autonomous driving. We demonstrated that the KL-Divergence penalty is crucial for further improving models that have been previously trained using expert demonstrations, as it prevents the new policy from forgetting previously learned tasks during training. Additionally, we incorporated a reward function tailored to address the weaknesses of the CIL++ model, as well as several other DRL techniques, which increased the generalization capability of CIL++ and its robustness to various scenarios. This approach opens up new pathways for addressing challenges such as generalization, offline learning biases, and environmental complexity in autonomous driving agents.

For future work, we plan to apply our methodology to further specific navigation tasks, such as Lane-Keeping Assistance, Lane-Centering, and Parking Assistance. Furthermore, we aim to explore

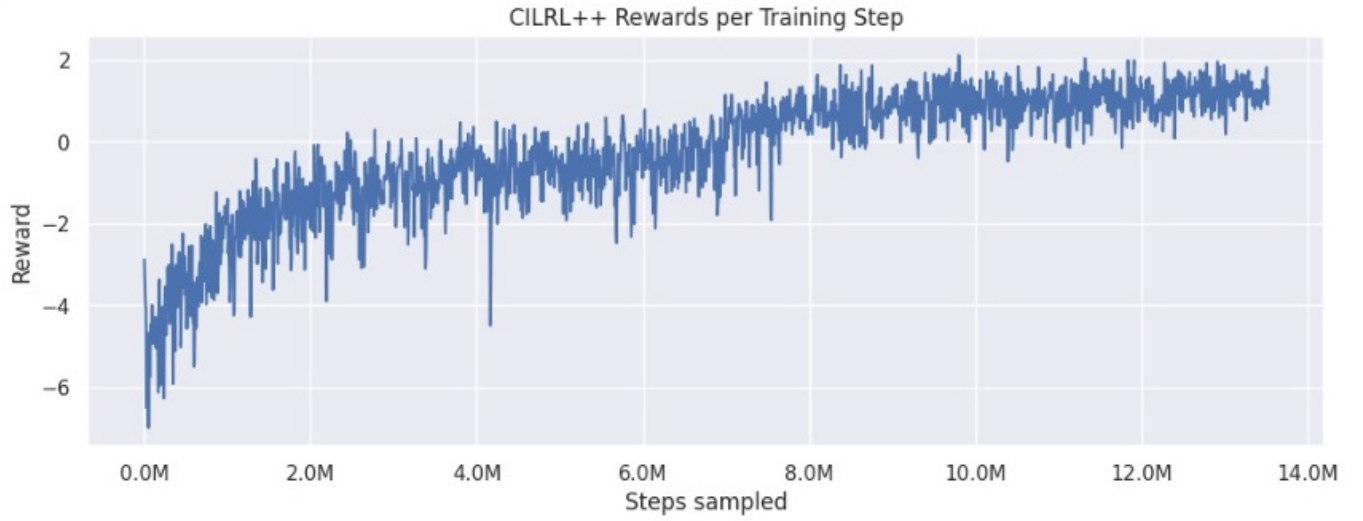


Figure 6: The performance of CILRL++ after 13M training steps.

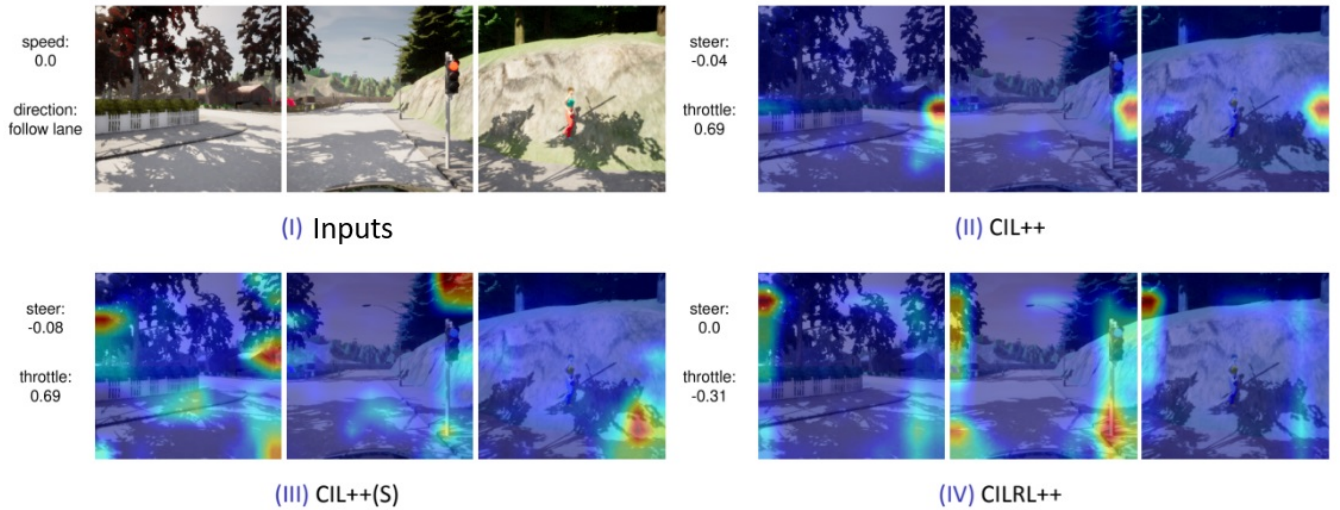


Figure 7: Action Explainability using Grad-Cam method. From the heatmap, it can be observed that CILRL++ correctly focuses on the traffic light and decelerates.

more sophisticated policy architectures by incorporating video inputs, which were not utilized in this study. Leveraging transformers could also be beneficial, as they have demonstrated outstanding performance in processing raw RGB images for navigation tasks. This could significantly enhance the model’s ability to understand and respond to complex driving environments, thereby improving the overall effectiveness and safety of autonomous driving systems.

REFERENCES

- [1] Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, Manu Orsini, Sertan Girgin, Raphael Marinier, Léonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, et al. 2020. What matters in on-policy reinforcement learning? a large-scale empirical study. *arXiv preprint arXiv:2006.05990* (2020).
- [2] Bowen Baker, Ilge Akkaya, Peter Zhokov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. 2022. Video pretraining (vpt): Learning to act by watching unlabeled online videos. *Advances in Neural Information Processing Systems* 35 (2022), 24639–24654.
- [3] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. 2020. End-to-end object detection with transformers. In *European conference on computer vision*. Springer, 213–229.
- [4] Karl W Cobbe, Jacob Hilton, Oleg Klimov, and John Schulman. 2021. Phasic policy gradient. In *International Conference on Machine Learning*. PMLR, 2020–2027.
- [5] Felipe Codevilla, Eder Santana, Antonio M López, and Adrien Gaidon. 2019. Exploring the limitations of behavior cloning for autonomous driving. In *Proceedings of the IEEE/CVF international conference on computer vision*. 9329–9338.
- [6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xi-aohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).

- [7] Aristotelis Lazaridis, Anestis Fachantidis, and Ioannis Vlahavas. 2020. Deep reinforcement learning: A state-of-the-art walkthrough. *Journal of Artificial Intelligence Research* 69 (2020), 1421–1471.
- [8] Xiaodan Liang, Tairui Wang, Luona Yang, and Eric Xing. 2018. Cirl: Controllable imitative reinforcement learning for vision-based self-driving. In *Proceedings of the European conference on computer vision (ECCV)*. 584–599.
- [9] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2015. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971* (2015).
- [10] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [11] Ramprasaath R Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. 2017. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision*. 618–626.
- [12] Richard S Sutton and Andrew G Barto. 2018. *Reinforcement learning: An introduction*. MIT press.
- [13] Ardi Tampuu, Tambet Matiisen, Maksym Semikin, Dmytro Fishman, and Naveed Muhammad. 2020. A survey of end-to-end driving: Architectures and training methods. *IEEE Transactions on Neural Networks and Learning Systems* 33, 4 (2020), 1364–1384.
- [14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [15] Yi Xiao, Felipe Codevilla, Diego Porres, and Antonio M López. 2023. Scaling Vision-Based End-to-End Autonomous Driving with Multi-View Attention Learning. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 1586–1593.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009